

PROYECTO

“PACKET SNIFFER” EN PYTHON

1 OBJETIVO

El objeto del proyecto es utilizar el lenguaje Python para realizar aplicaciones en red. En concreto se diseñará una aplicación para capturar paquetes de la red, y se discriminarán las diferentes cabeceras

- Cabecera IP
 - Cabecera ICMP.
 - Cabecera UDP
 - Cabecera DNS
 - Cabecera TCP

Si bien es importante realizar el diseño de este programa de forma individualizada para aprender las decodificaciones de todos los campos de las cabeceras de los diferentes protocolos, esta tarea estará incompleta si no se efectúan contrastaciones del correcto funcionamiento de la herramienta diseñada. Dicho de otra manera, una vez diseñado el programa hay que someterlo a pruebas que demuestren su correcto funcionamiento.

Para ello, será preciso forzar situaciones conocidas que permitan comprobar el correcto funcionamiento. Un ejemplo: Se está comprobando el protocolo ICMP y se quiere verificar lo que sucede cuando el TTL se agota (esto es, la recepción de un mensaje ICMP de error por agotamiento del TTL). Será preciso realizar el envío de una trama que contenga un Datagrama IP con un TTL pequeño (por ejemplo, utilizando el comando ping con el TTL elegido) y mejor aún si personalizamos todo aquello que se pueda (Dirección IP de destino, Número de bytes enviados, etc.) para reconocerlo posteriormente en la captura del paquete. Una buena forma de personalizar los paquetes a enviar puede ser mediante el uso de la herramienta “Colasoft Packet Builder”.

2 DESCRIPCIÓN

2.1 SNIFFER BÁSICO

2.1.1 SNIFFER MÍNIMO

1. Se utilizará PyScripter (ejecutar como administrador) como entorno de desarrollo integrado para editar y depurar los programas Sniffer.
2. En el manual de ayuda de Python aparece un pequeño ejemplo (ver Fig. 1) de un “sniffer”. Este será el programa de partida.
 - 2.1. Editar este programa en PyScripter

2.1.1. Para realizar capturas sencillas definiendo la dirección IP del adaptador de red, sustituir la línea de obtención del HOST por una asignación directa:

En los laboratorios: HOST = '10.11.61.xx' o HOST = '10.11.69.xx'.

En casa: Buscar la dirección IP del adaptador de red utilizado.

Típicamente: HOST = '192.168.1.xx'

2.2. Comprobar su funcionamiento

The next example shows how to write a very simple network **sniffer** with raw sockets on Windows. The example requires administrator privileges to modify the interface:

```
import socket

# the public network interface
HOST = socket.gethostbyname(socket.gethostname())

# create a raw socket and bind it to the public interface
s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
s.bind((HOST, 0))

# Include IP headers
s.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)

# receive all packages
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

# receive a package
print(s.recvfrom(65565))

# disabled promiscuous mode
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
```

Fig. 1 Ejemplo de un pequeño y simple programa sniffer (localizable en la ayuda de Python)

2.1.2 SNIFFER REPETITIVO

1. Sobre el programa anterior realizar modificaciones para que efectúe 10 capturas de forma secuencial
2. Para cada paquete capturado presentar los datos capturados en formato decimal, utilizando una estructura de cuatro columnas, separadas por espacios, en cada fila.

2.2 SNIFFER

2.2.1 OBTENCIÓN CABECERA IP

1. Separar la cabecera y los datos a nivel de datagrama IP
2. Obtener todos los campos de la cabecera IP (por separado)
3. Visualizar en consola los valores de cada campo, con un comentario apropiado
4. Comprobar su funcionamiento, enviando Datagramas IP a direcciones IP Destino conocidas, con longitudes específicas y con valores de TTL predefinidos (ping puede ayudarte con todo esto).

2.2.2 OBTENCIÓN DATOS IP

1. En función del protocolo de la cabecera del datagrama IP efectuar la discriminación de las cabeceras del siguiente nivel.

2.2.3 OBTENCIÓN DE LA CABECERA ICMP

2.2.3.1 DISCRIMINACIÓN DE ALGUNOS TIPOS DE MENSAJES ICMP

2.2.3.1.1 PETICIÓN DE ECO

2.2.3.1.2 RESPUESTA DE ECO

2.2.3.1.3 HOST NO ALCANZABLE

2.2.3.1.4 TIEMPO EXCEDIDO

2.2.4 OBTENCIÓN DE LA CABECERA DEL DATAGRAMA UDP

1. Separar la cabecera y los datos a nivel de datagrama UDP
2. Obtener todos los campos de la cabecera UDP (por separado)
3. Visualizar en consola los valores de cada campo, con un comentario apropiado
4. Obtener los datos UDP

2.2.4.1 OBTENCIÓN DE LA CABECERA DNS

2.2.4.2 OBTENCIÓN DE LOS DATOS DNS

2.2.5 OBTENCIÓN DE LA CABECERA DEL SEGMENTO TCP

1. Separar la cabecera y los datos a nivel de segmento TCP
2. Obtener todos los campos de la cabecera TCP (por separado)
3. Visualizar en consola los valores de cada campo, con un comentario apropiado
4. Obtener los datos TCP

3 ANEXO

3.1 FUNCIONES DE PYTHON

Se enumeran y describen algunas funciones de utilidad para capturar paquetes y decodificar la información transmitida en los mismos.

3.1.1 `SOCKET.RECVFROM(BUFSIZE[, FLAGS])`

Receive data from the socket. The return value is a pair (bytes, address) where *bytes* is a bytes object representing the data received and *address* is the address of the socket sending the data. See the Unix manual page *recv(2)* for the meaning of the optional argument *flags*; it defaults to zero. (The format of *address* depends on the address family — see above.)

3.1.2 `STRUCT.UNPACK(FMT, BUFFER)`

Unpack from the buffer *buffer* (presumably packed by `pack(fmt, ...)`) according to the format string *fmt*. The result is a tuple even if it contains exactly one item. The buffer must contain exactly the amount of data required by the format (`len(bytes)` must equal `calcsz(fmt)`).

Character	Byte order	Size	Alignment
@	native	native	native
=	native	standard	none
<	little-endian	standard	none
>	big-endian	standard	none
!	network (= big-endian)	standard	none

Fig. 2 Ordenamiento de los Bytes

Format	C Type	Python type	Standard size
<code>x</code>	pad byte	no value	
<code>c</code>	char	bytes of length 1	1
<code>b</code>	signed char	integer	1
<code>B</code>	unsigned char	integer	1
<code>?</code>	<code>_Bool</code>	bool	1
<code>h</code>	short	integer	2
<code>H</code>	unsigned short	integer	2
<code>i</code>	int	integer	4
<code>I</code>	unsigned int	integer	4
<code>l</code>	long	integer	4
<code>L</code>	unsigned long	integer	4
<code>q</code>	long long	integer	8
<code>Q</code>	unsigned long long	integer	8
<code>n</code>	<code>ssize_t</code>	integer	
<code>N</code>	<code>size_t</code>	integer	
<code>f</code>	float	float	4
<code>d</code>	double	float	8
<code>s</code>	char[]	bytes	
<code>p</code>	char[]	bytes	
<code>P</code>	void *	integer	

Fig. 3 Tipos y tamaños para cada formato

3.2 FRAGMENTO DE PROGRAMA DE TRATAMIENTO DE LOS DATOS CAPTURADOS

```
# receive a package
# socket.recvfrom(bufsize[, flags])
# Receive data from the socket.
# The return value is a pair (bytes, address) where
# bytes is a bytes object representing the data received and
# address is the address of the socket sending the data.
# See the Unix manual page recv(2) for the meaning of the optional argument flags;
# it defaults to zero. (The format of address depends on the address family – see above)

• for z in range (1,10):
•     print ()
•     print ('=====')
•     print ('Captura numero ', z)
•     data = s.recvfrom(65565)
•     packetdata = data[0]
•     addr      = data[1]

•     CabeceraIP = struct.unpack('!BBHHHBBBHHHBB', packetdata[:20])
•     datos = packetdata[20:]

•     print("Cabecera IP = ", CabeceraIP)
•     print("Datos = ")

•     TotalLength = CabeceraIP [2]
•     i2 = (TotalLength//4)
•     for i in (range(i2)):
•         print('{0:3d}: {1:4d} {2:4d} {3:4d} {4:4d}'.format (i, packetdata[4*i], packetdata[4*i+4], packetdata[4*i+8], packetdata[4*i+12]))

•     VersionIPandHeaderLength = CabeceraIP[0]
•     VersionIP = VersionIPandHeaderLength >> 4
•     print ("Version IP = ", VersionIP)
•     IHL = VersionIPandHeaderLength & 0x0F
•     print ("IHL = ", IHL)

•     DifferentiatedServices = CabeceraIP[1]
•     print ("Differentiated Services = ", DifferentiatedServices)
```

Fig. 4 Ejemplo de tratamiento de los datos capturados

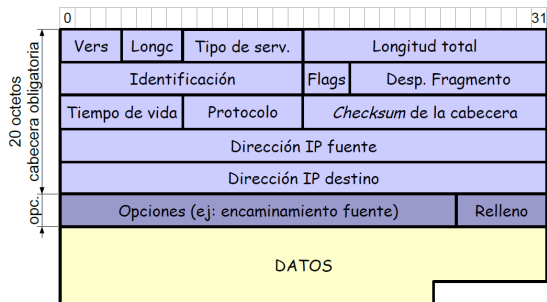


Fig. 1 Cabecera IP

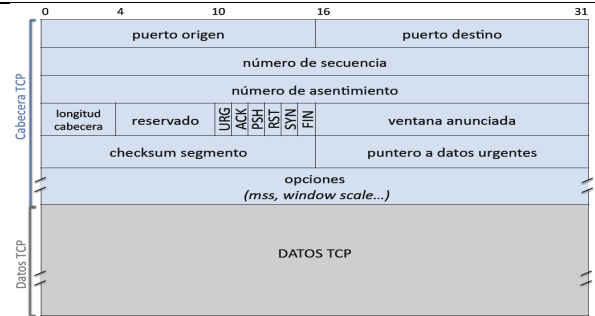


Fig. 2 Cabecera TCP

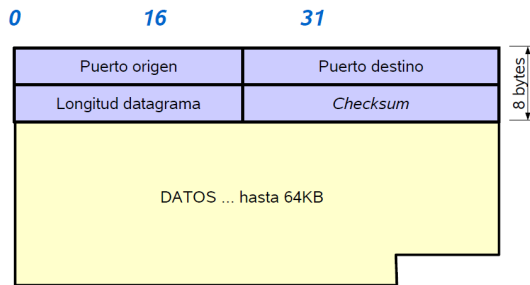


Fig. 3 Cabecera UDP

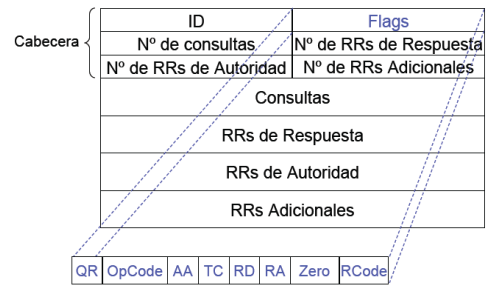


Fig. 4 Cabecera DNS + Datos

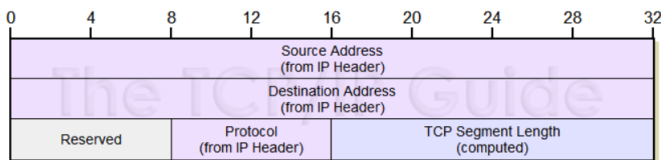


Fig. 5 Pseudo-cabecera TCP

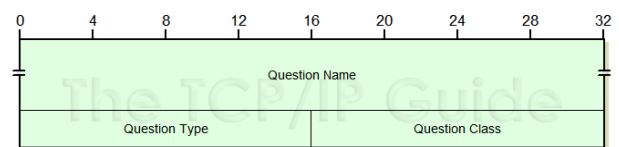


Figure 249: DNS Message Question Section Format

Fig. 6 Formato Sección Question del Mensaje DNS

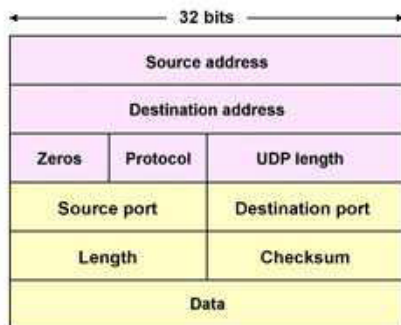


Fig. 7 Pseudo-cabecera UDP

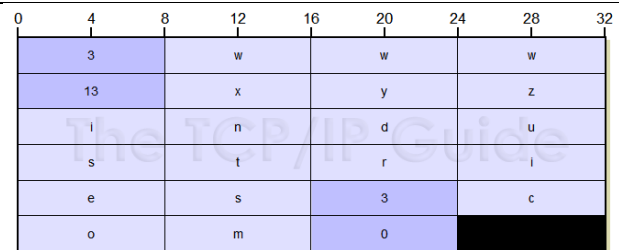


Figure 252: DNS Standard Name Notation

In DNS every named object or other name is represented by a sequence of label lengths and then labels, with each label length taking one byte and each label taking one byte per character. This example shows the encoding of the name "www.xyzindustries.com".

Fig. 8 Notación estándar de nombres DNS

ASCII Hex Simbolo	ASCII Hex Simbolo	ASCII Hex Simbolo	ASCII Hex Simbolo
0 0 NUL	16 10 DLE	32 20 (espacio)	48 30 0
1 1 SOH	17 11 DC1	33 21 !	49 31 1
2 2 STX	18 12 DC2	34 22 "	50 32 2
3 3 ETX	19 13 DC3	35 23 #	51 33 3
4 4 EOT	20 14 DC4	36 24 \$	52 34 4
5 5 ENQ	21 15 NAK	37 25 %	53 35 5
6 6 ACK	22 16 SYN	38 26 &	54 36 6
7 7 BEL	23 17 ETB	39 27 '	55 37 7
8 8 BS	24 18 CAN	40 28 (	56 38 8
9 9 TAB	25 19 EM	41 29)	57 39 9
10 A LF	26 1A SUB	42 2A *	58 3A :
11 B VT	27 1B ESC	43 2B +	59 3B ;
12 C FF	28 1C FS	44 2C ,	60 3C <
13 D CR	29 1D GS	45 2D -	61 3D =
14 E SO	30 1E RS	46 2E .	62 3E >
15 F SI	31 1F US	47 2F /	63 3F ?

ASCII Hex Simbolo	ASCII Hex Simbolo	ASCII Hex Simbolo	ASCII Hex Simbolo
64 40 @	80 50 P	96 60 `	112 70 p
65 41 A	81 51 Q	97 61 a	113 71 q
66 42 B	82 52 R	98 62 b	114 72 r
67 43 C	83 53 S	99 63 c	115 73 s
68 44 D	84 54 T	100 64 d	116 74 t
69 45 E	85 55 U	101 65 e	117 75 u
70 46 F	86 56 V	102 66 f	118 76 v
71 47 G	87 57 W	103 67 g	119 77 w
72 48 H	88 58 X	104 68 h	120 78 x
73 49 I	89 59 Y	105 69 i	121 79 y
74 4A J	90 5A Z	106 6A j	122 7A z
75 4B K	91 5B [	107 6B k	123 7B {
76 4C L	92 5C \	108 6C l	124 7C }
77 4D M	93 5D]	109 6D m	125 7D ~
78 4E N	94 5E ^	110 6E n	126 7E
79 4F O	95 5F _	111 6F o	127 7F

Fig. 9 Tabla de caracteres y códigos ASCII

Value Char	Value Char	Value Char	Value Char
0 A	16 Q	32 g	48 w
1 B	17 R	33 h	49 x
2 C	18 S	34 i	50 y
3 D	19 T	35 j	51 z
4 E	20 U	36 k	52 0
5 F	21 V	37 l	53 1
6 G	22 W	38 m	54 2
7 H	23 X	39 n	55 3
8 I	24 Y	40 o	56 4
9 J	25 Z	41 p	57 5
10 K	26 a	42 q	58 6
11 L	27 b	43 r	59 7
12 M	28 c	44 s	60 8
13 N	29 d	45 t	61 9
14 O	30 e	46 u	62 +
15 P	31 f	47 v	63 /

Fig. 10 Tabla de códigos base64

ICMP Echo Request/Response (Ping)

In a ping operation the entire packet is echo'd (or pinged as in ping-pong) back to the sender. A trace route comand (tracert) is usually a series of ping commands with increasing values of the TTL parameter (in IP header) such that it will be returned from each successive router.

Octet	Len	Name	Notes
0	1	ICMP Type	Message Type 8 = Echo request 0 = echo reply
1	1	Code	Code = 0
2-3	2	Checksum	-
4-5	2	Identifier	Used by sender to identify operation.
6-7	2	Sequence	Used by sender to identify operation.
8+	?	Data	Optional Data field.

ICMP Unreachable

The code field specifies the type of error.

Octet	Len	Name	Notes
0	1	ICMP Type	Message Type 1 = Host unreachable
1	1	Code	0 = Network unreachable 1 = Host unreachable 2 = Protocol unreachable 3 = Port unreachable 4 = Frag needed but DF set 5 = Source route failed 6 = Destination network unknown 7 = Destination host unknown 8 = Source host isolated 9 = Network access prohibited 10 = Host access prohibited 11 = Network unreachable for ToS 12 = Host unreachable for ToS
2-3	2	Checksum	-
4-5	2	Not used	Must be zero
6-7	2	Not used	Must be zero
8+	?	User Packet	IP header plus first 64 bits (8 octets) of failing datagram.

ICMP Redirect

Indicates the host should use the specified gateway to reach the IP address contained in the returned message.

Octet	Len	Name	Notes
0	1	ICMP Type	Message Type 5 = ICMP redirect
1	1	Code	May take one of the following values 0 = redirect datagrams for net (obsolete) 1 = redirect datagrams for host 2 = redirect datagrams for ToS and net 3 = redirect datagrams for Tos and host
2-3	2	Checksum	-
4-7	4	Gateway IP	Specifies that, for the destination host in the returned datagram, this gateway should be used.
8+	?	User Packet	IP header plus first 64 bits (8 octets) of failing datagram.

ICMP Time Exceeded

Message returned by the discovering router when the TTL count reaches 0 in the IP header or timeout problem with fragmentation.

Octet	Len	Name	Notes
0	1	ICMP Type	Message Type 11 = ICMP Time Exceeded
1	1	Code	May take one of the following values 0 = Time to Live count = 0 (exceeded) 1 = fragment reassembly time exceeded
2-3	2	Checksum	-
4-7	4	Unused	must be zero.
8+	?	User Packet	IP header plus first 64 bits (8 octets) of failing datagram.

Números de protocolos y puertos

Assigned Internet Protocol Numbers			Port Assignments:		
Decimal	Keyword	Protocol	Decimal	Keyword	Description
-----	-----	-----	-----	-----	-----
0		Reserved	0		Reserved
1	ICMP	Internet Control Message	1	TCPMUX	TCP Port Service Multiplexer
2	IGMP	Internet Group Management	2-4		Unassigned
3	GGP	Gateway-to-Gateway	5	RJE	Remote Job Entry
4		Unassigned	7	ECHO	Echo
5	ST	Stream	9	DISCARD	Discard
6	TCP	Transmission Control	11	USERS	Active Users
7	UCL	UCL	13	DAYTIME	Daytime
8	EGP	Exterior Gateway Protocol	15		Unassigned
9	IGP	any private interior gateway	17	QUOTE	Quote of the Day
10	BBN-RCC-MON	BBN RCC Monitoring	19	CHARGEN	Character Generator
11	NVP-II	Network Voice Protocol	20	FTP-DATA	File Transfer [Default Data]
12	PUP	PUP	21	FTP	File Transfer [Control]
13	ARGUS	ARGUS	23	TELNET	Telnet
14	EMCON	EMCON	25	SMTP	Simple Mail Transfer
15	XNET	Cross Net Debugger	27	NSW-FE	NSW User System FE
16	CHAOS	Chaos	29	MSG-ICP	MSG ICP
17	UDP	User Datagram	31	MSG-AUTH	MSG Authentication
18	MUX	Multiplexing	33	DSP	Display Support Protocol
19	DCN-MEAS	DCN Measurement Subsystems	35		any private printer server
20	HMP	Host Monitoring	37	TIME	Time
21	PRM	Packet Radio Measurement	39	RLP	Resource Location Protocol
22	XNS-IDP	XEROX NS IDP	41	GRAPHICS	Graphics
23	TRUNK-1	Trunk-1	42	NAMESERVER	Host Name Server
24	TRUNK-2	Trunk-2	43	NICNAME	Who Is
25	LEAF-1	Leaf-1			
26	LEAF-2	Leaf-2	44	MPM-FLAGS	MPM FLAGS Protocol
27	RDP	Reliable Data Protocol	45	MPM	Message Processing Module [recv]
28	IRTP	Internet Reliable Transaction	46	MPM-SND	MPM [default send]
29	ISO-TP4	ISO Transport Protocol Class 4	47	NI-FTP	NI FTP
30	NETBLT	Bulk Data Transfer Protocol	49	LOGIN	Login Host Protocol
31	MFE-NSP	MFE Network Services Protocol	51	LA-MAINT	IMP Logical Address Maintenance
32	MERIT-INP	MERIT Internodal Protocol	53	DOMAIN	Domain Name Server
33	SEP	Sequential Exchange Protocol	55	ISI-GL	ISI Graphics Language
34	3PC	Third Party Connect Protocol	57		any private terminal access
35-60		Unassigned	59		any private file service
61		any host internal protocol	61	NI-MAIL	NI MAIL
62	CFTP	CFTP	63	VIA-FTP	VIA Systems - FTP
			65	TACACS-DS	TACACS-Database Service
			67	BOOTPS	Bootstrap Protocol Server
			68	BOOTPC	Bootstrap Protocol Client
			69	TFTP	Trivial File Transfer
			71	NETRJS-1	Remote Job Service
			72	NETRJS-2	Remote Job Service
			73	NETRJS-3	Remote Job Service
			74	NETRJS-4	Remote Job Service
			75		any private dial out service
			77		any private RJE service
			79	FINGER	Finger